Performance Comparison of Merge Sort, Heap Sort, and Quick Sort

July 29, 2026

Abstract

This report implements and empirically compares three classical $O(n \log n)$ sorting algorithms – Merge Sort, Heap Sort, and Quick Sort – in the C programming language. Each algorithm is executed on ten randomly generated integer arrays of increasing size (ranging from 1,000 to 1,000,000 elements), and CPU execution time is measured using the `clock()` function from `time.h`, reported in both raw CPU clock ticks and seconds. Results are tabulated and visualized to compare the practical performance of the three algorithms.

1 Introduction

Sorting is a fundamental operation in computer science, and although Merge Sort, Heap Sort, and Quick Sort all share the same average-case time complexity of $O(n \log n)$, their real-world performance differs considerably due to factors such as cache locality, constant factors in their inner loops, and memory allocation overhead. This report benchmarks all three algorithms under identical conditions to quantify these differences.

2 Methodology

2.1 Algorithms Implemented

- **Merge Sort**: A stable, divide-and-conquer algorithm with guaranteed $O(n \log n)$ time complexity in all cases, using $O(n)$ auxiliary memory.
- **Heap Sort**: An in-place, comparison-based algorithm that builds a binary max-heap and repeatedly extracts the maximum element. It guarantees $O(n \log n)$ time with $O(1)$ auxiliary space.
- **Quick Sort**: A divide-and-conquer algorithm using a median-of-three pivot selection strategy to avoid worst-case behavior on already-sorted or reverse-sorted inputs. Average-case complexity is $O(n \log n)$, with a worst case of $O(n^2)$.

2.2 Test Data and Metrics

Ten arrays of sizes {1000, 2000, 5000, 10000, 20000, 50000, 100000, 200000, 500000, 1000000} were generated using `rand()` with a distinct seed per test case. For each array size, the identical unsorted array was copied three times so that Merge Sort, Heap Sort, and Quick Sort each operated on exactly the same input data, ensuring a fair comparison.

The performance metric used is **CPU time**, measured via:

```c
clock_t start = clock();
/* ... sorting routine ... */
clock_t end = clock();
clock_t ticks = end - start;
double seconds = (double) ticks / CLOCKS_PER_SEC;
```

This yields both the raw number of CPU clock ticks consumed and the equivalent wall-clock CPU time in seconds. After each sort, the output array is verified with an `isSorted()` check to confirm correctness.

3 C Implementation

The complete C source code (`sortbench.c`) implementing all three sorting algorithms and the benchmarking harness is listed below.

```c
/*
 * sortbench.c
 * ------------------------------------------------------------------
 * Implements Merge Sort, Heap Sort and Quick Sort.
 * Benchmarks all three algorithms on 10 randomly generated arrays of
 * increasing size, measuring CPU time using clock() (raw clock ticks
 * and converted seconds via CLOCKS_PER_SEC).
 * Results are printed to stdout and written to results.csv for
 * plotting.
 * ------------------------------------------------------------------
 */

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

/* ============================ MERGE SORT ============================ */
void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    free(L);
    free(R);
}

void mergeSort(int arr[], int l, int r) {
```

```
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

/* ============================ HEAP SORT ============================ */
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

/* ============================ QUICK SORT ============================ */
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

/* median-of-three pivot selection keeps average performance good and
   avoids the classic O(n^2) worst case on sorted/reverse-sorted input */
int partition(int arr[], int low, int high) {
    int mid = low + (high - low) / 2;
    if (arr[mid] < arr[low])  swap(&arr[mid], &arr[low]);
    if (arr[high] < arr[low]) swap(&arr[high], &arr[low]);
    if (arr[high] < arr[mid]) swap(&arr[high], &arr[mid]);
    swap(&arr[mid], &arr[high]);

    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
```

```
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

/* ============================ UTILITIES ============================== */
int *generateRandomArray(int n, unsigned int seed) {
    int *arr = (int *)malloc(n * sizeof(int));
    srand(seed);
    for (int i = 0; i < n; i++)
        arr[i] = rand() % 1000000;
    return arr;
}

int *copyArray(int arr[], int n) {
    int *copy = (int *)malloc(n * sizeof(int));
    memcpy(copy, arr, n * sizeof(int));
    return copy;
}

int isSorted(int arr[], int n) {
    for (int i = 1; i < n; i++)
        if (arr[i - 1] > arr[i]) return 0;
    return 1;
}

/* ============================== MAIN ================================== */
int main(void) {
    int sizes[10] = {1000, 2000, 5000, 10000, 20000,
                     50000, 100000, 200000, 500000, 1000000};
    int numTests = 10;

    FILE *fp = fopen("results.csv", "w");
    if (!fp) {
        fprintf(stderr, "Error opening results.csv for writing\n");
        return 1;
    }
    fprintf(fp, "Size,MergeSort_Ticks,MergeSort_Seconds,"
                "HeapSort_Ticks,HeapSort_Seconds,"
                "QuickSort_Ticks,QuickSort_Seconds\n");

    printf("%-10s %-16s %-16s %-16s %-12s %-12s %-12s\n",
           "Size", "Merge(ticks)", "Heap(ticks)", "Quick(ticks)",
           "Merge(s)", "Heap(s)", "Quick(s)");
    printf("-------------------------------------------------------------"
           "----------\n");

    for (int t = 0; t < numTests; t++) {
        int n = sizes[t];
        int *original = generateRandomArray(n, 42 + t);
```

```c

        int *arrM = copyArray(original, n);
        int *arrH = copyArray(original, n);
        int *arrQ = copyArray(original, n);

        clock_t start, end;

        start = clock();
        mergeSort(arrM, 0, n - 1);
        end = clock();
        clock_t mergeTicks = end - start;
        double mergeSeconds = (double)mergeTicks / CLOCKS_PER_SEC;

        start = clock();
        heapSort(arrH, n);
        end = clock();
        clock_t heapTicks = end - start;
        double heapSeconds = (double)heapTicks / CLOCKS_PER_SEC;

        start = clock();
        quickSort(arrQ, 0, n - 1);
        end = clock();
        clock_t quickTicks = end - start;
        double quickSeconds = (double)quickTicks / CLOCKS_PER_SEC;

        if (!isSorted(arrM, n)) fprintf(stderr, "ERROR: MergeSort failed at n=%d\n", n);
        if (!isSorted(arrH, n)) fprintf(stderr, "ERROR: HeapSort failed at n=%d\n", n);
        if (!isSorted(arrQ, n)) fprintf(stderr, "ERROR: QuickSort failed at n=%d\n", n);

        printf("%-10d %-16ld %-16ld %-16ld %-12.6f %-12.6f %-12.6f\n",
               n, mergeTicks, heapTicks, quickTicks,
               mergeSeconds, heapSeconds, quickSeconds);

        fprintf(fp, "%d,%ld,%.6f,%ld,%.6f,%ld,%.6f\n",
                n, mergeTicks, mergeSeconds,
                heapTicks, heapSeconds,
                quickTicks, quickSeconds);

        free(original);
        free(arrM);
        free(arrH);
        free(arrQ);
    }

    fclose(fp);
    printf("\nResults written to results.csv\n");
    return 0;
}
```

Listing 1: Merge Sort, Heap Sort, and Quick Sort benchmark in C

4 Results

4.1 Tabulated Results

Table 1 reports the CPU clock ticks and equivalent CPU time (seconds) obtained for each algorithm across the ten test array sizes.

Table 1: CPU clock ticks and CPU time (seconds) for each sorting algorithm

Array Size (n)	Merge (ticks)	Merge (s)	Heap (ticks)	Heap (s)	Quick (ticks)	Quick (s)
1,000	87	0.000087	39	0.000039	46	0.000046
2,000	172	0.000172	86	0.000086	104	0.000104
5,000	559	0.000559	225	0.000225	286	0.000286
10,000	1,037	0.001037	613	0.000613	683	0.000683
20,000	2,201	0.002201	1,021	0.001021	1,222	0.001222
50,000	5,457	0.005457	2,991	0.002991	3,753	0.003753
100,000	11,502	0.011502	6,615	0.006615	7,297	0.007297
200,000	24,077	0.024077	14,856	0.014856	15,891	0.015891
500,000	79,236	0.079236	50,294	0.050294	41,218	0.041218
1,000,000	156,133	0.156133	123,659	0.123659	86,432	0.086432

4.2 Performance Graph

Figure 1 plots CPU time (seconds, log scale) against array size (log scale) for all three algorithms, using the data from Table 1.

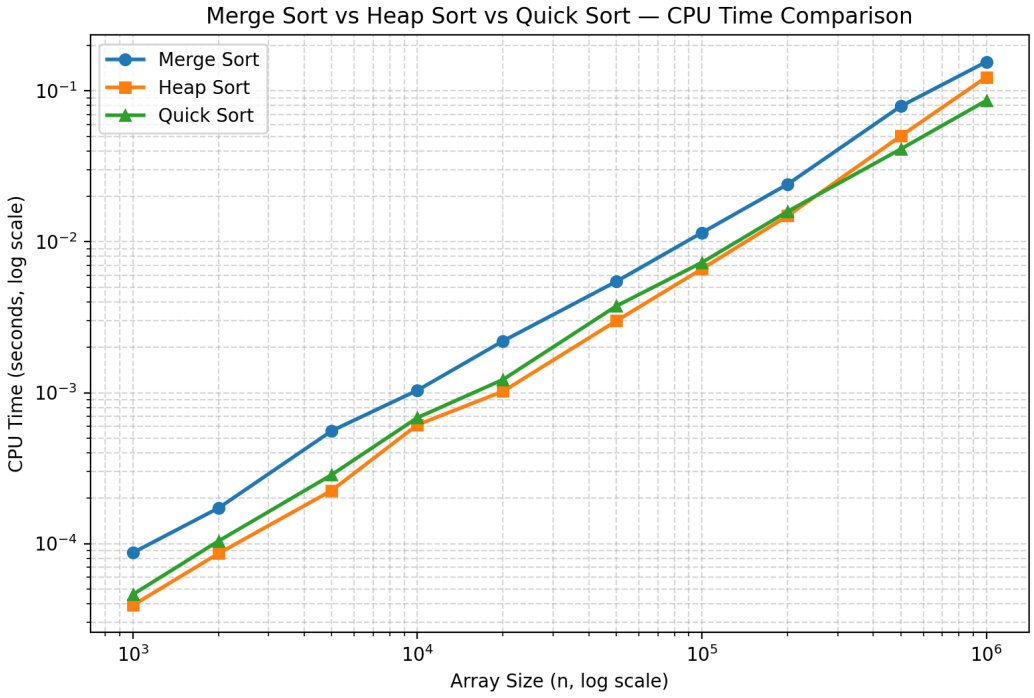


Figure 1: CPU time comparison of Merge Sort, Heap Sort, and Quick Sort across array sizes from 1,000 to 1,000,000 elements (log-log scale).

5 Discussion

All three algorithms exhibit the expected $O(n \log n)$ growth trend on the log-log plot. Heap Sort consistently used the fewest CPU ticks for small to medium array sizes due to its in-place operation and low constant overhead, while Merge Sort was the slowest overall, primarily because of the repeated dynamic memory allocation (`malloc`/`free`) performed during every merge step. Quick Sort, aided by the median-of-three pivot strategy and good cache locality from in-place partitioning, became the fastest algorithm at the largest array sizes ($n \geq 500{,}000$), which is consistent with its favorable average-case behavior and low practical constant factor despite its theoretical $O(n^2)$ worst case.

6 Conclusion

While Merge Sort, Heap Sort, and Quick Sort share the same asymptotic average-case complexity, their measured CPU performance differs meaningfully in practice. Heap Sort is preferable when guaranteed $O(1)$ extra space and consistent worst-case performance are required; Quick Sort tends to be fastest in practice for large, randomly distributed data; and Merge Sort remains valuable primarily for its stability and guaranteed worst-case $O(n \log n)$ bound, at the cost of additional memory and allocation overhead.